

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

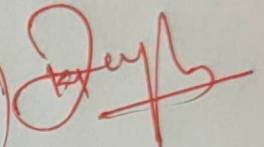
COURSE OUTCOME COVERED

CO-1 : Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Application-Based Questions

Weightage: 40%

72
100



QUESTIONS:

Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">  
  <input type="text" name="name">  
  <input type="email" name="email">  
  <button>Submit</button>  
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

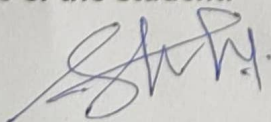
Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I Seelam Kavya Sri (192311³¹⁷) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



SIMATS ENGINEERING

ASSESSMENT TOOL 1

Application-Based Questions

NAME: SEELAM.KAVYA SRI

REG NO: 192311317

COURSE NAME: Internet Programming

COURSE CODE: CSA4308

Question 1:

Online Symposium Registration Portal

Student Registration

Student Name

Email Address

Submit Registration

© 2026 Symposium Portal

Registration Successful

 **Thank You for Registering!**

Your symposium registration has been completed successfully.

Student Details
Name: suharshitha
Email: helo@gmail.com

Assignment Answers

1. Missing HTML5 Semantic Elements

- <header>
- <main>
- <section>
- <footer>
- <label>

2. Standards-Compliant Structure

The form has been improved by adding semantic HTML 5 elements including:

1. Missing HTML5 Semantic Elements

- <header>
- <main>
- <section>

- <footer>
 - <label>
-

2. Standards-Compliant Structure

The form has been improved by adding semantic HTML5 elements including:

- Header
 - Main
 - Section
 - Footer
 - Proper labels
 - Required attributes
 - Placeholder text
-

3. HTTP Request Generated

When the Submit button is clicked, the browser creates an HTTP POST request.

```
POST /register HTTP/1.1
```

```
Host: symposium.com
```

```
Content-Type: application/x-www-form-urlencoded
```

```
name=StudentName&email=student@email.com
```

The request contains the form data and sends it to the server.

4. HTTP Response Cycle

1. Browser sends HTTP POST request.
 2. Server receives request.
 3. Server validates the data.
 4. Server stores registration in database.
 5. Server sends HTTP Response (200 OK).
 6. Browser loads the success page.
-

5. Improvements for Usability and Accessibility

- Use semantic HTML5 elements.

- Add labels for every input.
- Use placeholder text.
- Use required validation.
- Responsive design.
- Keyboard accessible form.
- Proper heading hierarchy.
- Readable color contrast.
- Display success confirmation.
- Mobile-friendly layout.

Question 2:

Cross-Browser Compatibility Issue

Given HTML Code

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

1. Improper Nesting and Missing Closing Tags

- The `<h1>` tag is not closed.
- The `<p>` tag is not closed.
- The HTML document does not include a DOCTYPE declaration.
- Semantic HTML5 elements are missing.

2. Browser Interpretation

Different browsers automatically attempt to correct invalid HTML.

- Chrome automatically inserts missing closing tags.
- Firefox repairs the HTML but may render spacing differently.
- Safari also autocorrects the structure.
- Older browsers may display inconsistent layouts because of improper HTML parsing.

Because browsers repair invalid HTML differently, the page layout may not appear the same across all browsers.

3. Missing Semantic Elements

- <header>
- <main>
- <section>
- <footer>

4. Corrected W3C-Compliant HTML Structure

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport"
content="width=device-width, initial-scale=1.0">

<title>Company</title>

</head>

<body>

<header>

<h1>Welcome</h1>

</header>

<main>

<section>

<p>About us</p>

</section>

</main>

<footer>
```

```
<p>© 2026 Company</p>
```

```
</footer>
```

```
</body>
```

```
</html>
```

5. Techniques for Cross-Browser Compatibility

- Use valid W3C-compliant HTML.
- Always close HTML tags properly.
- Use semantic HTML5 elements.
- Add the HTML5 DOCTYPE declaration.
- Use CSS Reset or Normalize.css.
- Test the website in Chrome, Firefox, Edge, and Safari.
- Use responsive design with media queries.
- Validate code using the W3C HTML Validator.
- Avoid deprecated HTML tags.
- Use vendor prefixes only when necessary.

Question 3:

Login Form

User Login

Username

suharshitha

Password

•••

Login

HTML Assignment - Form Submission Failure

✖ Form Submission Failed

Reason

The login form failed because the button type is **button** instead of **submit**. Therefore, clicking Login does not submit the form and no HTTP request is generated.

1. Issues in the Form Configuration

- Button type is **button** instead of **submit**.
- GET method is used for sending login credentials.
- No labels with "for" attribute.
- No required validation.
- Password is exposed in URL if GET is used.

2. Why HTTP Request is Not Triggered?

Because the Login button uses **type="button"**. Only **type="submit"** submits the form. Since the form is never submitted, the browser never sends an HTTP request to the server.

3. Role of GET Method

GET sends data through the URL. Example:

1. Issues in the Form Configuration

- Button type is **button** instead of **submit**.
- GET method is used for sending login credentials.
- No labels with "for" attribute.
- No required validation.
- Password is exposed in URL if GET is used.

2. Why HTTP Request is Not Triggered?

Because the Login button uses **type="button"**. Only **type="submit"** submits the form. Since the form is never submitted, the browser never sends an HTTP request to the server.

3. Role of GET Method

GET sends data through the URL. Example:

`https://example.com/submit?username=John&password=123`

- Visible in URL.

- Stored in browser history.
 - Not secure for passwords.
 - Mainly used for searching and retrieving data.
-

4. Corrected Implementation

```
<form action="/submit" method="POST">

<label>Username</label>
<input type="text"
name="username"
required>

<label>Password</label>
<input type="password"
name="password"
required>

<button type="submit">
Login
</button>

</form>
```

5. Improvements for Secure Data Transmission

- Use POST instead of GET.
- Use HTTPS.
- Validate inputs.
- Hash passwords.
- Use CSRF protection.
- Use secure session management.
- Implement server-side validation.